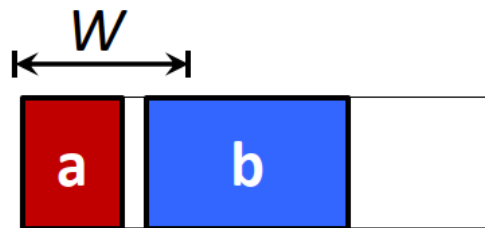


1. Refer to p.5 to p.9, unit 4, why is filler cell insertion alone no longer sufficient to fix all MinIA constraint violations in modern technology nodes? Please provide two example scenarios and explain why filler cell insertion cannot resolve the violations in each case.

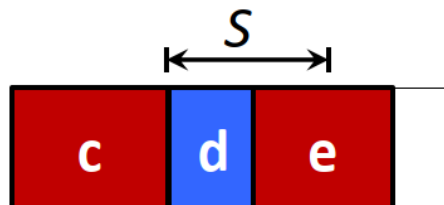
在講義第四章第 9 頁提到，下面兩種情況無法單靠 Filler cell insertion 解決

(1) Minimum implant width



a 的寬度小於 Minimum implant width W ，a、b 為不同類型的 Implant regions，且兩者之間空間不夠大的情況下，即使做了 Filler cell insertion 也無法讓 a 的寬度滿足 W ，因此單靠 Filler cell insertion 無法解決問題。

(2) Minimum spacing



c 和 d 為同類型的 Implant regions，兩者之間的空隙小於 Minimum spacing S ，且還有卡著不同類型的 Cell d，因此 c 和 e 無法直接用 Filler cell insertion 來做 Merging，所以單靠 Filler cell insertion 永遠無法讓空隙滿足 S 。

2. Refer to p.27 to p.38, unit 4.

- a. If there are K cells in a row, then how many nodes and edges are in the graph model when both adjacent cell swapping and cell flipping can be applied?

這邊不考慮 Source node s 和 Target node t

(1) Nodes 的數量 = $6K - 4$

對於 Node $_i$ 來說，一定會有 O_i 、 f_i 兩種選擇。若 $K \geq i+1$ ，Node $_i$ 可以與 Node $_{i+1}$ 做 Swapping，多出 $O_{i+1} O_i$ 、 $O_{i+1} f_i$ 、 $f_{i+1} O_i$ 、 $f_{i+1} f_i$ 四種選擇。所以除了 Node $_K$ 僅兩種選擇，其餘皆有六種，Nodes 的數量為 $6K - 4$

(2) Edges 的數量:

$$4 \cdot (K-1) + 8 \cdot (K-2) + 8 \cdot (K-2) + 16 \cdot (K-3) \quad K \geq 3$$

$$4 \quad K = 2$$

$$0 \quad K = 1$$

Node 有分成單點 (沒做 Swapping) 和雙點 (有做 Swapping)，一般情況下，Edge 可以分成四類：

a. 單點 $\rightarrow$ 單點

$O_i / f_i \rightarrow O_{i+1} / f_{i+1}$ ，共有 $4 \cdot (K-1)$ 條邊

b. 單點 $\rightarrow$ 雙點

$O_i / f_i \rightarrow O_{i+2} O_{i+1} / O_{i+2} f_{i+1} / f_{i+2} O_{i+1} / f_{i+2} f_{i+1}$ ，共有 $8 \cdot (K-2)$ 條邊

c. 雙點 $\rightarrow$ 單點

$O_{i+1} O_i / O_{i+1} f_i / f_{i+1} O_i / f_{i+1} f_i \rightarrow O_{i+2} / f_{i+2}$ ，共有 $8 \cdot (K-2)$ 條邊

d. 雙點 $\rightarrow$ 雙點

$O_{i+1} O_i / O_{i+1} f_i / f_{i+1} O_i / f_{i+1} f_i \rightarrow O_{i+3} O_{i+2} / O_{i+3} f_{i+2} / f_{i+3} O_{i+2} / f_{i+3} f_{i+2}$ ，共有 $16 \cdot (K-3)$ 條邊

當 $K = 1$ 時，沒有任何 Edge，Edge 數量為 0

當 $K = 2$ 時，僅存在 單點 $\rightarrow$ 單點，Edge 數量為 $4 \cdot (2-1) = 4$

當 $K \geq 3$ 時，Edge 數量為 $4 \cdot (K-1) + 8 \cdot (K-2) + 8 \cdot (K-2) + 16 \cdot (K-3)$

- b. When solving the DDA minimization problem as the shortest path problem considering both adjacent cell swapping and cell flipping, how would you modify the edge cost to minimize $\alpha * \text{\#cell flipping} + \beta * \text{\#cell swapping}$ ($\alpha, \beta \in \mathbb{Z}^+$ and) while ensuring that the total number of the DDA between adjacent cells is still minimized?

目標是在最小化 D2D 數量的情況下，盡量讓 $\alpha \cdot \text{\#cell flipping} + \beta \cdot \text{\#cell swapping}$ 也越小越好

所以要先把 D2D 發生的 Cost 定的很大，我設定為 $\gamma = (\alpha + \beta) \cdot n + 1$ ， n 為 Node 的數量，代表的意義是即使在最壞的情況下，所有能做的 Swapping 和 Flipping 都發生，Cost 仍然不會比 D2D 大。

整體的 Cost 會變成

$$O_i, \text{Cost} = 0$$

$$f_i, \text{Cost} = \alpha$$

$$O_{i+2} O_{i+1}, \text{Cost} = \beta$$

$$O_{i+2} f_{i+1}, \text{Cost} = \alpha + \beta$$

$$f_{i+2} O_{i+1}, \text{Cost} = \alpha + \beta$$

$$f_{i+2} f_{i+1}, \text{Cost} = 2\alpha + \beta$$

$$\text{D2D 發生時, Cost 再加上 } (\alpha + \beta) \cdot n + 1$$

3. Refer to p.6, unit 5.

- a. What features are extracted from layout patterns to support machine learning based hotspot detection in [Yu+DAC13]? For each feature, briefly describe its definition and explain why it may help detect hotspots.

根據 *Machine-Learning-Based Hotspot Detection Using Topological Classification and Critical Feature Extraction*，取出的 Features 有分成兩種：

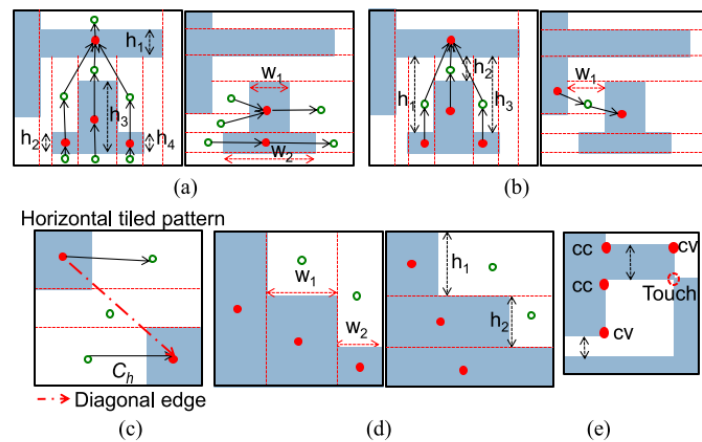


Fig. 7. Critical features. (a) Internal feature. (b) External feature. (c) Diagonal feature. (d) Segment feature. (e) Nontopological feature.

(1) Topological Features

- a. Internal feature:

The Width and Height of a Block Tile

Block Tile 太小、太窄的話很容易變成 Hotspot

- b. External Feature:

The Distance Between Two Adjacent Block Tiles

Block tiles 之間間距太小也容易變成 Hotspot

- c. Diagonal Feature:

The Diagonal Relations (or Distance) Between Two Convex Corners of Block Tiles (Space Tiles)

對角的距離如果太近也容易變成 Hotspot

- d. Segment Feature:

The Space Tile With Two or Three Edges Touching the Window Boundary or Space Tiles

如果 Layout 中的 Pattern 剛好出現在偵測視窗的邊邊，模型看到

的 Pattern 可能不完整產生誤判，錯把沒問題的 Pattern 當成 hotspot，或漏掉真正的 hotspot

(2) Non-Topological Features

a. Number of corners

Corner 越多 Pattern 越複雜，容易變形出現 Hotspot

b. Number of touched points

Touched points 代表 Pattern 貼得比較緊，密度高，越多越容易出現 Hotspot

c. The minimum distance between a pair of internally facing polygon edges

內部相鄰 Edge 太近容易出現 Hotspot

d. The minimum distance between a pair of externally facing polygon edges

外部相鄰的 Pattern 太近容易出現 Hotspot

e. The polygon density

高密度區域容易出現 Hotspot

這些 Features 都和 Hotspot 出現的機率有高度相關

b. Suggest and briefly describe one additional feature not mentioned in the paper and explain why this feature might be useful for identifying hotspots.

這篇論文有用到 Polygon Density 作為模型的 Feature，可以把它當成這整個區域的 Global Density。但如果在這個區域內的左側很密集，右側都是空白的，Polygon Density 依然是中間值，卻無法表達內部分布的情況。

我認為可以加入 Local Density Variation 作為 Feature，像是把整個區域分成 3x3 個子區域，分別計算各自的 Local Density，再計算 Local Density Variation 來看整個區域的 Pattern 分布均不均勻，越不均勻越可能成為 Hotspot。

4. Refer to p.22, unit 6, a serial edge simplification rule is used to eliminate a degree-2 node in the flipping graph. The updated gain between nodes i and k is defined as

$$gain_{i,k} = \max(gain_{i,j}, gain_{j,k}) - \max(gain_{i,j} + gain_{j,k}, 0)$$

where j is the degree-2 node connected to i and k . Derive this equation and explain the reasoning behind each term.

當 $Node_i$ 、 $Node_k$ 的顏色 (Mask) 相同時，會因為 $Node_j$ 的顏色不同產生兩種可能：Cut 兩邊的 Edge 以及兩邊都不 Cut。GREMA 會選擇 Gain (Number of stitches reduced) 較高的一方，因此 Flipping Gain 為 $\max(gain_{i,j} + gain_{j,k}, 0)$ 。

當 $Node_i$ 、 $Node_k$ 的顏色不同時，也會因為 $Node_j$ 的顏色不同產生兩種可能：Cut 任何一條 Edge。同樣的，GREMA 會選擇 Gain 較高的那條，因此 Flipping Gain 為 $\max(gain_{i,j}, gain_{j,k})$ 。

至於為什麼 $gain_{i,k}$ 是 $\max(gain_{i,j}, gain_{j,k}) - \max(gain_{i,j} + gain_{j,k}, 0)$ 是因為一開始預設 $Node_i$ 、 $Node_k$ 顏色是相同的，因此 $\max(gain_{i,j} + gain_{j,k}, 0)$ 就成為當前已經獲得的 Gain 值。

$gain_{i,k}$ 代表改變 $Node_i$ 、 $Node_k$ 其中一個所獲得的 Gain 值，因此當改變發生時，必須扣掉原本已經獲得的 $\max(gain_{i,j} + gain_{j,k}, 0)$ ，加上最新得到的 $\max(gain_{i,j}, gain_{j,k})$ ，最後結果是 $\max(gain_{i,j}, gain_{j,k}) - \max(gain_{i,j} + gain_{j,k}, 0)$ ，也就是 $gain_{i,k}$ 的值。

5. Refer to p.28, unit 6, briefly describe how [Kahng+ ICCAD08] address the DPL decomposition problem (10 points), and why GREMA outperforms [Kahng+ ICCAD08] in terms of solution quality (10 points) and runtime (10 points).

[Kahng+ ICCAD08] 主要分成四步：

- (1) Graph Construction: 建立 Layout 的 Conflict Graph
- (2) Conflict Cycle Detection: 找到 Odd-length cycle (無法用 2-color 處理的結構)
- (3) Node Splitting: 為了解決 Conflict Cycle，要持續做 Node splitting 把某些大 Feature 拆成數個小 Feature 直到沒有 Conflict Cycle 為止
- (4) Color Assignment via ILP: 解決所有 Conflict Cycle 後，用 ILP 解決 2-color 的問題

Solution Quality

當 Stitches 的數量上升，良率會下降，因此這邊的 Solution Quality 是以 Stitches 的數量為指標，數量越低 Quality 越好。

對於 Conflict Graph，GREMA 會先生成 Candidate Stitches 來解決所有 Conflict Cycle，轉換成 Flipping Graph 後，再透過 Max-cut on Flipping Graph 來選擇哪些 Candidate Stitches 真的需要保留，而不是像 [Kahng+ ICCAD08] 保留全部 Stitches 直接解一輪 ILP。這樣篩選的機制可以大幅減少 Stitches 的數量，因此 GREMA 的 Solution Quality 會比較好。

Runtime

GREMA 是利用 Flipping Graph 的概念，把原本很複雜的 Conflict Graph 一堆 Patterns 精簡成更少量的 Clusters，再透過 Serial edge simplification、Parallel edge simplification 或 Partition propagation 等方法大幅減少 Max-cut 問題的 Input Size。

GREMA 和 [Kahng+ ICCAD08] 都用 ILP 解決問題，但前者的 Input Size 小非常多，很多時候甚至不需要 ILP (簡化後剩下 2 個 nodes 的情況)，因此 GREMA 的 Runtime 會更低，效率更高。